

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**СИСТЕМА ЗА ОБРАБОТКА НА ОБЛАЦИ ОТ
ТОЧКИ И КАРТИ НА ДЪЛБОЧИНАТА С
УСКОРЕНО ТЪРСЕНЕ ВЪРХУ
ПРОСТРАНСТВЕН ИНДЕКС**

КУРСОВ ПРОЕКТ

по дисциплина „Компютърно зрение“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Милена Лазарова

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА	2
II. ПРОЕКТИРАНЕ НА СИСТЕМАТА	4
III. ПРОГРАМНА РЕАЛИЗАЦИЯ	5
IV. МЕТОДИКА НА ЕКСПЕРИМЕНТАЛНАТА ПРОВЕРКА	7
V. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	8
ЗАКЛЮЧЕНИЕ	11
ЛИТЕРАТУРА	12
ПРИЛОЖЕНИЕ	13

УВОД

Сензорите за дълбочина и стереоскопичните камери извеждат *облак от точки*: неопределено множество от координати, чиито съседства не се четат от индекса на пиксела, а трябва да бъдат намерени. Почти всяка стъпка от обработката, от премахването на шума през оценяването на нормалите до съвместяването на два припокриващи се записа, се свежда до един и същ въпрос: кои точки лежат близо до дадена точка. Затова цената на конвейера се определя не толкова от аритметиката на отделния алгоритъм, колкото от скоростта на отговора.

Целта е конвейер за обработка на облаци от точки и растерни изображения, в който търсенето на съседи е изнесено в отделен компонент върху пространствен индекс, и всяка стъпка е измерима самостоятелно. Реализацията е на C++20, без нито една външна зависимост. Ускорението идва от подредба на данните по компонента и от пакетно устроен вътрешен цикъл, който компилаторът превежда във векторни инструкции; че това се случва, се проверява от отчета на компилатора, а не се предполага.

Задачите са: преглед на предметната област и на класическите методи; обосновка на избора на индекс, на стратегия за векторизация и на организация на конвейера; реализация на стъпките, тоест вход и изход, прореждане с вокселна решетка, индекс със заявки за най-близки съседи и по радиус в два взаимозаменяеми варианта на вътрешния цикъл, съвместяване по схемата на итеративната най-близка точка и растерна верига от изглаждане, градиент, потискане на немаксимуми и сегментация; методика за измерване спрямо независим еталон; и анализ на измерванията.

Обхватът е геометричната и фотометричната обработка. Извън него остават оценяването на дълбочина по стереодвойка и извличането на описания на околност, и двете потребители на заявката за съседи, чиято цена се измерва пряко тук. Извън обхвата съзнателно остават и разпознаването по обучени модели и невронните мрежи. Причината е несъвместима методика на оценяване: качеството на обучен разпознавател зависи преди всичко от набора от данни и от процедурата на обучение, а предмет на измерване тук е цената на геометричната обработка. Тези теми са разгледани в литературния преглед, но не са реализирани и авторът не претендира за опит в тях.

I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА

1.1. Теоретична рамка

Системата за компютърно зрение е верига от източник на данни, предварителна обработка, извличане на признаци и вземане на решение. Апаратната част определя дефектите на входа: пасивната стереодвойка дава плътна карта на дълбочината само там, където сцената има текстура, а активните сензори дават плътност, независима от текстурата, но внасят шум по краищата и загуба на данни при отразяващи повърхности. Информационната структура следва от този избор: подреденото изображение на дълбочината запазва съседството по индекс, докато облакът от точки го губи и налага пространствен индекс.

Предварителната обработка цели потискане на шума без разрушаване на геометрията. Линейното изглаждане размива именно ръбовете, които следващите стъпки търсят, затова се предпочитат методи, съобразени с ръба: двустранното филтриране претегля съседите едновременно по разстояние и по разлика в стойността [1]. Откриването на контури следва постановката на Кани с трите ѝ критерия за добро откриване, добра локализация и единичен отговор [2]; върху карта на дълбочината същият апарат откроява геометричните прекъсвания, но изисква отделно третиране на липсващите стойности.

Извличането на признаци разделя подредените от неподредените данни. За облаци от точки утвърдено средство са хистограмите на бързите точкови признаци, които описват околността чрез разпределение на ъглите между нормалите на съседите [3]. Сегментацията се води или от геометрията, чрез съгласуваност на нормалите и кривината, или от подгонване на параметричен модел, което почти винаги стъпва върху устойчиво оценяване със случайни извадки [4]. Текстурната сегментация ползва статистиките от матрицата на съвместната поява на нива на сивото [5].

Възстановяването на дълбочината по стереодвойка се свежда до задача за съответствие; полуглобалното съгласуване я решава чрез сумиране на едномерни динамични програми по няколко направления [6]. Когато сцената се сменя от няколко положения, облаци трябва да се приведат в обща координатна система. Каноничното решение е итеративната най-близка точка [7], чиито варианти се различават по избора на съответствия, по метриката и по стратегията на отхвърляне, и тези решения влияят на скоростта на сходимост повече от самата формулировка [8]. Всяка итерация изисква

търсене на най-близък съсед, което връща изложението към структурата на данните: k-d дървото [9] остава основното средство, а приблизителните варианти купуват скорост срещу контролирана загуба на точност [10]. Разпознаването по модел заема последното ниво и е извън реализацията; сегментираните области и описанията на околностите са точно входът, който един разпознавател очаква. Утвърдените реализации с отворен код [11, 12] служат за ориентир кои алгоритми са установената практика.

1.2. Разгледани решения и направен избор

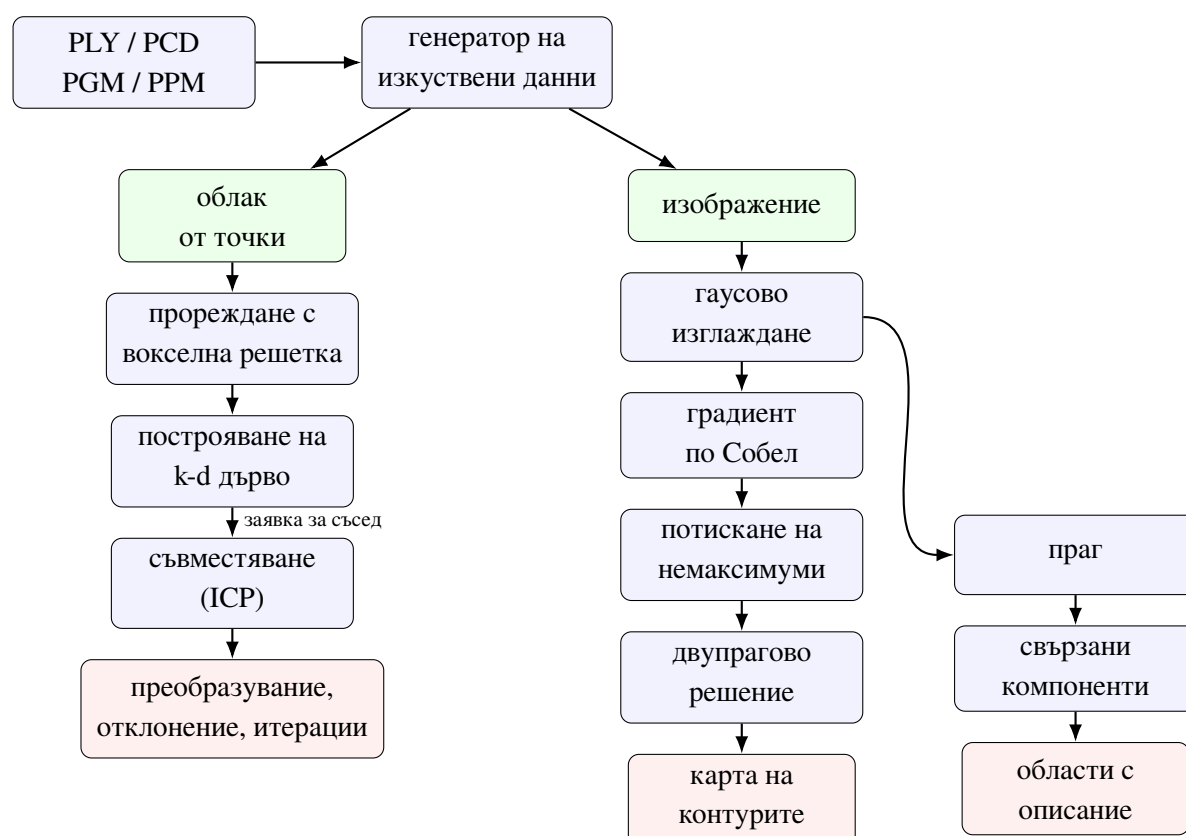
Пространствен индекс. Вокселната решетка е тривиална за построяване и бърза при равномерна плътност, но при неравномерна плътност или изразходва памет за празни кубчета, или събира прекалено много точки в едно. K-d дървото [9] се приспособява към плътността, защото разделя по медиана, но обхождането му е с нередовен достъп до паметта, а октодървото стои между двете. Избрано е k-d дървото, а вокселната решетка се използва в другата си роля, като стъпка за прореждане преди индексирването. Приблизителните схеми [10] са извън обхвата, защото внасят втори източник на грешка, който би се смесил с грешката на измервания алгоритъм.

Векторизация. Вътрешните функции на конкретен набор инструкции дават пълен контрол, но обвързват изходния текст с една архитектура. Библиотека за преносима векторизация запазва единен изходен текст, но е външна зависимост. Автоматичната векторизация е най-евтина, но зависи от компилатора. Избран е третият път, с добавка, която премахва главния му недостатък: вътрешният цикъл е написан нарочно в пакетна форма, която компилаторът разпознава, и че я разпознава, се проверява с `-fopenmp` (Раздел III). Условието изборът да работи е координатите да са подредени по компонента, а не по точка, което превръща оформлението на паметта в проектно решение (Раздел II).

Конвейер и външни библиотеки. Монолитната функция е най-кратка, но прави отделната стъпка неизмерима, затова всяка стъпка е обект с еднакъв договор, който приема облак и връща облак заедно с показатели за изпълнението си. За външните библиотеки бяха разгледани две положения: те да поемат входа и изхода и да служат за еталон [11, 12], или проектът да няма нито една задължителна външна зависимост. Избрано е второто, по практически довод: изграждане, което изисква мениджър на пакети, на практика не се изпълнява от никого при първия опит. Цената е платена в собствен код за четците, проверката, измерването и разлагането на матрица 3×3 .

II. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Системата е на четири слоя: представяне на данните, пространствен индекс, стъпки на обработката и приложения. Всяка стъпка зависи само от двата долни слоя и връща нов резултат, без да променя входа си на място, така че отпада скрита зависимост по ред на изпълнение (Фиг. 1).



Фигура 1. Поток на данните. Геометричният клон е отляво, растерният отдясно.

Разположение в паметта. Облакът се съхранява като три непрекъснати масива от float, по един на координата, а не като масив от тройки, защото вътрешният цикъл на обхождането на лист чете една координата последователно: крачката е 4 байта вместо 12 и едно четене на кеш-линия обслужва шестнадесет точки.

Интерфейс на индекса. Двата пътя за обхождане на лист, *скаларен* и *пакетен*, не са две реализации, а параметър на една и съща заявка. Решението е методическо: така двете измервания се различават единствено по тялото на цикъла.

Извън обхвата. Отпадат втора реализация на индекса чрез вокселна решетка (сравнението щеше да повтори извода на Раздел I), сравнението с чужда библиотека (заради нулевите зависимости), стереодвойката и описанията на околност (по обем).

III. ПРОГРАМНА РЕАЛИЗАЦИЯ

Изходният текст е на C++20 и се изгражда с CMake 3.20 или по-нова. Разпределението по файлове и адресът на хранилището са в Приложението. Тук са отбелязани само местата, които не са очевидни от кода.

Построяване и обхождане на индекса. Разделянето на всяко ниво се прави с `std::nth_element` по медианата на *най-дългата* ос на текущото подмножество. Редуването на осите по нива дава дълги тесни клетки при сцена, в която една ос е много по-къса от другите, а тогава разстоянието до разделящата равнина рядко надхвърля текущото най-добро и отсичането не работи. Координатите се разбъркват в реда на обхождане чак накрая, наведнъж, така че точките на един лист са съседни в паметта; това е предпоставката пакетният път да има смисъл. Ако всички точки в подмножеството имат еднаква стойност по избраната ос, разделянето по медиана не напредва и рекурсията не свършва: проверката е дали медианата съвпада едновременно с първия и с последния елемент след `nth_element`, и при съвпадение се създава лист, дори по-голям от зададения размер. Случаят не е измислен, защото облак, легнал точно върху равнина, го предизвиква по оста, по която е плосък. Обхождането е итеративно, със собствен стек. Кандидатите се държат в купчина върху вектор, подаден от повикващия и преизползван между заявките, защото съвместяването прави милиони заявки и заделянето на памет за всяка от тях доминира над самото търсене.

Двата пътя на обхождане на лист. Това е мястото, около което е построена цялата работа. Скаларният път обхожда точките една по една и след всяка проверява дали кандидатът влиза в купчината. Пакетният път разделя същата работа на два цикъла. Първият смята квадратите на разстоянията за цял блок от точки в масив на стека: без разклонения, без обръщение към купчината, без зависимост между итерациите, с три последователни четения. Точно това е формата, която компилаторът разпознава. Вторият цикъл прави проверките и обновяванията. Между двата стои проверка, която не е оптимизация на дребно: ако купчината е пълна и дори най-близката точка от блока е по-далече от най-лошата приета, целият блок се пропуска.

Че цикълът се векторизира, е проверено, а не предположено. При компилиране на `src/kdtree.cpp` с `-O3 -fopenmp -fopenmp-simd` отговорът за редовете на пакетния път е `loop vectorized using 16 byte vectors`, тоест четири числа с плаваща запетая наведнъж,

a с `-march=native` на изпитваната машина същите редове дават 32 byte vectors. Редовете на скаларния път не се появяват в отчета при никоя от двете настройки.

Разлагане и съвместяване. Вместо библиотека за линейна алгебра е написан едностранен метод на Якоби за разлагане на матрица 3×3 . Той е предпочетен пред разлагане на $A^T A$, защото последното повдига числото на обусловеност на квадрат, а тук матрицата е кръстосана ковариация между два облака и при почти плосък облак вече е зле обусловена. Едно място се оказва източник на действителна грешка: когато матрицата е с ранг под три, съответният стълб на U не е определен от входа и нормата му излиза нула. Първата реализация оставяше на негово място единичен вектор, което прави U неортогонална, а произведението VU^T тогава понякога излиза *отражение*, не ротация. Проверката `icp.kabsch_never_returns_a_reflection`, която подава точки върху равнина, го улови; поправката допълва стълба до ортонормиран базис чрез векторно произведение на другите два. Сумирането на центровете на тежестта е в `double`, защото при облак, отместен далече от началото на координатната система, натрупването във `float` губи точно значещите цифри, които носят формата.

Всяка итерация на съвместяването намира най-близък съсед за всяка точка от преместения изходен облак, отхвърля съответствията над зададен праг, оценява преобразуване и го прилага. Прагът не е украса: без него точките без покритие в целевия облак също получават съответствие и издърпват решението към грешен отговор. Оценената добавка се смята върху *вече преместения* облак, затова се композира отляво към натрупаното преобразуване; обратният ред дава решение, което изглежда правдоподобно, но не съвпада с еталона.

Прореждане и растерни стъпки. Групирането по кубчета става чрез *сортиране* на 64-битови ключове, а не чрез хеш-таблица, заради повторемостта: хеш-таблицата дава ред на изхода, зависещ от реализацията, а две изпълнения върху един вход трябва да дадат еднакъв изход. Изходната точка на кубче е центърът на тежестта, а не средата, която внася грешка до половин ребро дори при точки върху равнина. Гаусовото ядро се нормира по *действителната* сума на отрязаното ядро, защото опашката е малка, но не е нула, и нормирането по аналитичната константа прави изображението системно по-тъмно точно с нея. Границата се обработва със захващане на координатите към ръба, не с допълване с нула, което би вкарало изкуствен контур по периметъра.

IV. МЕТОДИКА НА ЕКСПЕРИМЕНТАЛНАТА ПРОВЕРКА

Измерването отговаря на два въпроса: дава ли реализацията същия резултат като еталон, за който е известно, че е верен, и колко време и памет отнема. Двете не се смесват, защото по-бърза стъпка с различен резултат е друга стъпка.

Еталон. За заявките към индекса еталонът е *изчерпателното търсене*, което е точно по определение и е реализирано в проекта; сравнението е по множеството от разстояния, не по номерата, защото при равни разстояния изборът на съсед не е част от договора. За съвместяването еталонът е *известното преобразуване*, приложено от генератора върху целевия облак, а за останалите стъпки е свойство, което резултатът трябва да има независимо от реализацията, например че гаусовото ядро сумира до единица.

Данни. Не се използва външен набор от данни; всичко се произвежда от генератора по зададено зърно, така че две изпълнения дават един и същ вход. За индекса се ползва равномерен облак в куб със страна 10 при 10 000, 50 000, 200 000 и 1 000 000 точки, защото при равномерност дървото е най-близо до балансирано. За съвместяването се ползва сцена от равнина, сфера и куб, общо 70 000 точки. Само равнина не стига, защото там съвместяването е недоопределено по две трансляции и по една ротация и „сходи“ към произволен член на цяло семейство решения, което изглежда като успех, докато не се сравни с еталона. Изходният облак е извадка от 75 % от точките, зашумена и преместена, тоест няма общи точки с целевия.

Машина и протокол. Всички числа са получени на една машина: AMD Ryzen 5 3600 (Zen 2), 6 ядра и 12 нишки, до 3950 MHz, L3 32768 KB, 15,9 GB оперативна памет, AVX2 без AVX-512, Windows 11 Pro N, g++ 15.2.0 под MinGW-w64, CMake 4.3.2 и Ninja 1.13.2, с флагове `-std=c++20 -O3 -Wall -Wextra`. Втората конфигурация е същата плюс `-march=native`; тя не е преносима и се отчита отделно, за да се види колко от печалбата идва от ширината на векторния регистър. Сравненията между двата пътя са с 9 повторения и 2 загрявания, останалите с 5 и 1, и се правят върху едно и също дърво и едни и същи заявки.

Отчита се **минимумът** от повторенията, а медианата и разсейването стоят до него: при първите изпълнения разсейването достигаше стотици проценти, тоест по ядрата има чужда работа, а външна намеса може само да *удължи* изпълнението.

V. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Числата идват от `results/measurements_baseline.txt` за подразбиращата се конфигурация и `results/measurements_native.txt` за тази с `-march=native`.

Съвпадение с еталона. Всички 60 проверки минават. Заявката за k най-близки съседни съпада с изчерпателното търсене по множеството от разстояния при $k = 1, 5, 32$, заявката по радиус съпада по множеството от номера, а дърво с лист 1 дава същия отговор като дървета с лист 4, 16, 64 и 256, включително върху изродени облаци. Разлагането възстановява входа с норма на разликата под 10^{-10} , оценяването по Кабш възстановява зададеното преобразуване с ъгъл под 10^{-5} rad, а кръговратът през PLY, PCD, PGM и PPM връща входа до 10^{-5} . Съпадението на *двама пътя* е с допуск точно нула, защото те извършват едни и същи операции в един и същ ред, само групирани различно. Проверката `icp.kabsch_never_returns_a_reflection` улови действителна грешка в разлагането (Раздел III) и показва защо еталонът трябва да е независим: реализацията минаваше проверката за възстановяване на матрицата, а грешеше само при вход с ранг под три.

Построяване на индекса. При размер на листа 32 построяването отнема 1,379 ms за 10 000 точки, 8,440 ms за 50 000, 50,659 ms за 200 000 и 432,459 ms за 1 000 000, при заета памет съответно 0,18, 0,88, 3,53 и 17,64 MiB, тоест 18,5 байта на точка. Времето расте по-бързо от линейното, но последното измерване е с разсейване 59,3 % и е най-ненадеждното от четирите.

Таблица 1. Заявка за k най-близки съседни, 20 000 заявки, 9 повторения, подразбираща се конфигурация

Точки	k	Скал., ms	Пак., ms	Отн.	Разс., %	Заявки/s
10 000	1	8,70	8,48	1,03	16,2	2 357 823
10 000	8	26,81	25,13	1,07	54,7	795 909
10 000	32	85,13	90,71	0,94	171,2	220 495
50 000	1	12,20	11,40	1,07	169,1	1 753 986
50 000	8	111,40	41,47	2,69	215,4	482 243
50 000	32	132,98	107,22	1,24	61,9	186 535
200 000	1	17,76	18,50	0,96	103,4	1 080 818
200 000	8	134,36	50,07	2,68	147,9	399 463
200 000	32	136,71	148,32	0,92	107,9	134 841
1 000 000	1	31,44	31,95	0,98	48,8	626 047
1 000 000	8	65,09	58,88	1,11	38,5	339 655
1 000 000	32	161,22	139,85	1,15	40,6	143 006

Табл. 1 не позволява извод за отношението между двата пътя и това е самият резултат. Отношението се движи между 0,92 и 2,69, а разсейването в същите редове стига 215 %. Двата реда с отношение около 2,69 са придружени от най-голямото разсейване, тоест там минимумът на скаларния път не е уловил неразтревожено изпълнение, и не се тълкуват; същите случаи с `-march=native` дават 0,88 и 1,13. Изводът, който данните поддържат, е отрицателен и е записан като такъв: *разликата между двата пътя в цялата заявка е под разделителната способност на тази машина*. Пропускателната способност все пак се чете: около 2,4 милиона заявки в секунда при $k = 1$ върху 10 000 точки и около 143 000 при $k = 32$ върху милион точки.

Таблица 2. Изолирано обхождане на лист, 9 повторения. Ляво: подразбираща се конфигурация. Дясно: с `-march=native`

Точки в листа	k	Подразбираща се			-march=native		
		Скал., ms	Пак., ms	Отн.	Скал., ms	Пак., ms	Отн.
1 024	1	93,76	37,18	2,52	54,12	23,77	2,28
1 024	8	109,85	82,44	1,33	91,70	70,46	1,30
8 192	1	55,79	26,16	2,13	53,95	17,72	3,05
8 192	8	62,63	30,99	2,02	57,23	24,68	2,32
65 536	1	52,91	22,23	2,38	49,87	15,30	3,26
65 536	8	42,97	18,38	2,34	55,25	23,24	2,38

Табл. 2 премахва обхождането на дървото от измерването: дървото е един лист и всяка заявка е само обхождане на лист. Тук резултатът е еднозначен. Пакетният път е между 2,0 и 2,5 пъти по-бърз в подразбиращата се конфигурация и между 2,3 и 3,3 пъти при `-march=native`, ако се изключи редът с $k = 8$ и лист от 1 024 точки, а посоката е една и съща във всичките дванадесет клетки. Печалбата от по-широкия регистър е подредена: при лист от 8 192 и 65 536 точки отношението нараства от 2,13 и 2,38 на 3,05 и 3,26, но не е двукратна, защото вторият цикъл, този с проверките и купчината, не се векторизира. Редът с лист 1 024 и $k = 8$ се отличава с отношение около 1,3, защото там купчината се пълни и остава пълна.

Радиус, размер на листа, прореждане. За заявката по радиус отношението между двата пътя се движи между 0,76 и 1,49 и не подкрепя извод; за нея това е и очакваното, защото скъпата част не е сметката на разстоянието, а записването на намерените съседи, а то е еднакво и за двата пътя. При 200 000 точки и $k = 8$ двете конфигурации се съгласяват за най-добрия размер на листа, 16 точки, с 8,71 ms и 8,63 ms за пакетния път и 8,86 ms

и 8,80 ms за скаларния. Прореждането на милион точки отнема между 107 и 150 ms за ребра от 0,250 до 0,020, докато броят изходни точки пада от 995 925 на 64 000: времето е доминирано от сортирането, което се прави винаги върху целия вход.

Таблица 3. Съвместяване, точка към точка. Праг за отхвърляне 0,5, най-много 60 итерации. Зададеното преобразуване е ротация от 7,45° и трансляция с дължина 0,229

Шум σ	Итерации	Сходимост	Отклонение	Грешка по ъгъл, °	Мин., ms
0,000	38	да	0,015230	0,0055	309,01
0,002	39	да	0,015365	0,0016	326,28
0,005	41	да	0,017720	0,0035	380,62
0,010	49	да	0,022771	0,0040	468,24
0,020	60	не	0,030791	0,0430	641,42
0,040	60	не	0,038900	0,8019	829,56

Броят итерации расте с шума и при $\sigma \geq 0,020$ границата от 60 се изчерпва преди сходимост. Грешката по ъгъл остава под 0,006° до $\sigma = 0,010$, тоест до шум, равен на една пета от реброто на прореждащата решетка, а при $\sigma = 0,040$ скача на 0,80°, двеста пъти повече, макар отклонението да е нараснало само 2,6 пъти. Отклонението при $\sigma = 0$ не е нула, а 0,0152, защото двата облака съдържат *различни* точки.

Растерен конвейер. Върху кадър 1024 на 768 гаусовото изглаждане при $\sigma = 1,6$ отнема 47,305 ms, градиентът по Собел 16,375 ms, потискането на немаксимумите 11,225 ms, двупраговото решение 0,707 ms и свързаните компоненти 5,925 ms, тоест между 0,9 и 60,2 ns на пиксел. Изглаждането доминира: то прави 22 умножения на пиксел, а градиентът 12 плюс по един корен. Отношението на броя умножения е 1,8, а измереното отношение на времената е 2,9; разликата не се обяснява тук, защото причината ѝ не е измерена.

Анализ. Първо, векторизацията на обхождането на лист работи и печалбата ѝ е около двукратна. Второ, тази печалба не се вижда в цялата заявка, защото обхождането на дървото, работата с купчината и записът на резултата не се векторизират. Това е отклонение от очакваното в Раздел I, където разположението по компонента беше избрано именно заради заявката: изборът не е погрешен, но частта, която покрива, е по-малка от предполагащото. Трето, съвместяването възстановява зададеното преобразуване с грешка под 0,006° при шум до една пета от реброто на решетката. Колко точно е печалбата от векторизацията в цялата заявка остава без отговор: тя не е нула, но е под разделителната способност на машината. Отговорът изисква машина без чужд товар, а не друга реализация. Числото липсва и не се замества с предположение.

ЗАКЛЮЧЕНИЕ

Проектът реализира конвейер за обработка на облаци от точки и растерни изображения, в който търсенето на съседи е обособено като отделен компонент върху пространствен индекс и е ускорено чрез подредба на данните по компонента и пакетно обхождане на листата. Обособяването има две последици. Първата е инженерна: всяка стъпка ползва един и същ тесен интерфейс. Втората е методическа: двата пътя на обхождане са параметър на една и съща заявка, не два различни класа, и затова се сравняват върху едно и също дърво, в едно и също изпълнение, при единствена разлика тялото на цикъла. Реализацията се компилира с компилатор за C++20 и CMake и с нищо друго; проверката, измерването и разлагането на матрица 3×3 са написани в проекта именно за да няма задължителна външна зависимост.

Измерените резултати са три и единият е отрицателен. Векторизацията на обхождането на лист работи: при изолирано измерване пакетният път е между 2,0 и 2,5 пъти по-бърз в подразбиращата се конфигурация и между 2,3 и 3,3 пъти при пълния набор инструкции, с еднаква посока във всички дванадесет клетки (Табл. 2). Тази печалба обаче не се вижда в цялата заявка: Табл. 1 не позволява извод, защото разсейването надхвърля търсената разлика. Съвместяването възстановява зададеното кораво преобразуване с грешка по ъгъл под $0,006^\circ$ при шум до една пета от реброто на прореждащата решетка (Табл. 3).

Ограниченията са три и всичките са известни. Печалбата от разположението по компонента зависи от вида на натоварването и е неизгодна за четене на единични точки в произволен ред. Измерванията са проведени на машина с чужд товар по ядрата; затова се отчита минимумът, а разсейването стои до всяко число, и времената при разсейване над сто процента не са надеждни. Разпознаването по обучени модели остава извън обхвата, разгледано в литературния преглед, но нереализирано и неизмерено.

Посоките за продължение са четири: повтаряне на измерването на цялата заявка на машина без страничен товар, което е единственото липсващо, за да получи вторият резултат количествен отговор; паралелно изпълнение върху няколко ядра, за което договорът на стъпката вече е подходящ; приблизително търсене на съседи с контролирана загуба на точност [10]; и добавяне на разпознавател над изхода на конвейера, което би затворило веригата от сензор до решение.

ЛІТЕРАТУРА

- [1] C. Tomasi and R. Manduchi, “Bilateral filtering for gray and color images,” in *Proc. IEEE ICCV*, pp. 839–846, 1998.
- [2] J. Canny, “A computational approach to edge detection,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 8, no. 6, pp. 679–698, 1986.
- [3] R. B. Rusu, N. Blodow, and M. Beetz, “Fast point feature histograms (FPFH) for 3D registration,” in *Proc. IEEE ICRA*, pp. 3212–3217, 2009.
- [4] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [5] R. M. Haralick, K. Shanmugam, and I. Dinstein, “Textural features for image classification,” *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-3, no. 6, pp. 610–621, 1973.
- [6] H. Hirschmüller, “Stereo processing by semiglobal matching and mutual information,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 30, no. 2, pp. 328–341, 2008.
- [7] P. J. Besl and N. D. McKay, “A method for registration of 3-D shapes,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 14, no. 2, pp. 239–256, 1992.
- [8] S. Rusinkiewicz and M. Levoy, “Efficient variants of the ICP algorithm,” in *Proc. Int. Conf. 3-D Digital Imaging and Modeling*, pp. 145–152, 2001.
- [9] J. L. Bentley, “Multidimensional binary search trees used for associative searching,” *Commun. ACM*, vol. 18, no. 9, pp. 509–517, 1975.
- [10] M. Muja and D. G. Lowe, “Scalable nearest neighbor algorithms for high dimensional data,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 36, no. 11, pp. 2227–2240, 2014.
- [11] R. B. Rusu and S. Cousins, “3D is here: Point Cloud Library (PCL),” in *Proc. IEEE ICRA*, pp. 1–4, 2011.
- [12] G. Bradski, “The OpenCV library,” *Dr. Dobb’s J. Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

ПРИЛОЖЕНИЕ

Целият изходен текст, записаните изпълнения на измерванията в `results/` и този документ са в публичното хранилище <https://github.com/Alex-Tsvetanov/pointforge>. Табл. 4 дава разпределението на кода; броят редове е за заглавния файл и реализацията взети заедно. Изгражда се с `cmake -S . -B build -DCMAKE_BUILD_TYPE=Release` и `cmake --build build`, след което `ctest --test-dir build` пуска проверките, целта `demo` произвежда изходните файлове, а `pointforge_bench --csv <файл>` повтаря измерванията. Втората конфигурация се получава с `-DPOINTFORGE_NATIVE_ARCH=ON` и не е преносима, затова е изключена по подразбиране.

Таблица 4. Модули на реализацията

Файл	Редове	Съдържание
<code>point_cloud.hpp</code>	153	облак от точки, разположен по компонента
<code>transform.{hpp,cpp}</code>	305	матрица 3x3, сингулярно разлагане, кораво преобразуване
<code>kdtree.{hpp,cpp}</code>	605	построяване, трите вида заявка, двата пътя на лист
<code>icp.{hpp,cpp}</code>	257	съответствия, отхвърляне, оценяване по Кабш
<code>voxel_grid.{hpp,cpp}</code>	149	прореждане с вокселна решетка
<code>cloud_io.{hpp,cpp}</code>	307	четене и запис на PLY и PCD
<code>image.{hpp,cpp}</code>	179	двата растерни типа и преобразуванията между тях
<code>image_io.{hpp,cpp}</code>	163	четене и запис на PGM и PPM
<code>image_ops.{hpp,cpp}</code>	238	изглаждане, градиент, немаксимуми, прагове
<code>labeling.{hpp,cpp}</code>	179	свързани компоненти и описанията им
<code>synthetic.{hpp,cpp}</code>	272	генератор на изкуствени облаци и изображения
<code>timing.hpp</code>	80	измерване на време без външна библиотека
<code>text.hpp</code>	37	подравняване на изхода при кирилица
<code>apps/demo.cpp</code>	256	демонстрация: една команда, видим изход
<code>benchmarks/bench.cpp</code>	417	измервания, изход и във формат CSV
<code>tests/</code>	1 163	рамка и шест групи проверки, 60 случая